My Parivaar – Administrator Module

Below is a **detailed, structured Product Documentation** for the 🛠️ **Administrator Module of *My Parivaar*.**

> ⚠️ **Note (Transparency)**
>
> The currently available internal document [My Parivaar - First Cut.page]() focuses on overall product vision and technology stack and **does not explicitly define an Administrator Module** .

>

> ✅ Therefore, the documentation below is a **proposed / recommended Administrator Module design**, aligned with the My Parivaar concept, scalable architecture, and best practices for family-centric platforms.

🛠️ My Parivaar – Administrator Module

Product Documentation (Admin Console)

🌟 1. Overview

The **Administrator Module** is the **control center** of **My Parivaar**, enabling authorized admins to **manage users, families, content, configurations, security, and platform health** from a single interface.

🎯 **Primary Goals**

- Ensure platform **safety & trust**
- Enable **smooth onboarding & governance**
- Provide **full visibility & control**
- Support **scalable growth**

👥 2. Admin Roles & Access Control 🔐

🔑 Supported Roles

	Role	Description
1	🟢 **Super Admin**	Full platform access, system-level control
2	🔵 **Admin**	User & content management
3	🟡 **Moderator**	Reviews content, handles reports

4	🟣 **Support Admin**	Assists users, manages tickets

✅ Role-based access ensures **least privilege principle**

🧭 3. Admin Dashboard 📊

The dashboard provides a **real-time snapshot** of the platform.

📌 Key Widgets

- 👪 Total Families Registered
- 👤 Active vs Inactive Users
- 🆕 New Registrations (Daily / Weekly)
- 🚨 Flagged Content & Reports
- 📈 Platform Usage Trends
- ⚙️ System Health Indicators

🎨 Visualized via charts, counters, and alerts

👤 4. User Management Module

✅ Core Capabilities

- 🔍 Search users by name, email, phone
- 👀 View user profiles (read-only sensitive fields)
- 🔒 Activate / Deactivate users
- 📜 View login history & activity logs
- 🔁 Reset user access (non-password-based)

🧠 Smart Insights

- Identify inactive users
- Detect suspicious behavior patterns
- Track engagement level

👪 5. Family & Relationship Management

Admins can manage **family units** within My Parivaar.

🏠 Features

- View family trees 🌳
- Validate or remove duplicate members
- Merge family groups (with audit trail)
- Handle disputes in family relationships
- Assign or revoke family admin roles

✅ Maintains **data accuracy & family integrity**

📄 6. Content Moderation & Governance 🛡️

🧹 Moderated Content

- Profile photos 🖼️
- Family posts 📝
- Comments 💬
- Shared memories 📸

🚨 Reporting Workflow

1. User reports content
2. Moderator reviews flagged item
3. Action taken:
   - ✅ Approve
   - ⚠️ Warn user
   - ❌ Remove content
   - 🚫 Suspend account

🕵️ All actions are **logged for audit**

⚙️ 7. Configuration & Settings Module

🧩 Platform Controls

- Feature toggles (enable/disable modules)
- Family size limits
- Invite & referral rules
- Notification settings 🔔
- Localization & language options 🌍

🔄 Version Management

- Rollout features gradually
- Enable beta testing groups

🔐 8. Security & Compliance Module

🔒 Security Controls

- Role-based access enforcement
- Session & token monitoring
- Login anomaly detection
- Multi-factor authentication (future-ready)

📜 Compliance

- Data access logs

- User consent tracking
- GDPR / privacy-aligned actions
- Soft-delete & retention policies

✅ Built for **trust & regulatory readiness**

🧠 9. Analytics & Reports 📈

📊 Available Reports

- User growth trends
- Family engagement metrics
- Feature adoption
- Content moderation stats
- Admin activity audit logs

📥 Export formats:

- CSV
- PDF
- Dashboard snapshots

🎧 10. Support & Issue Management

🆘 Admin Support Tools

- View user-raised tickets
- Tag issues (bug / feature / misuse)
- Assign tickets to support admins
- Track resolution status
- Internal notes for collaboration

📨 Integrated notifications for faster response

🧩 11. Audit Logs & Observability 👁️

Every admin action is tracked.

📜 Logged Actions

- User status changes
- Content moderation decisions
- Role assignments
- Configuration updates

✅ Immutable logs ensure **accountability**

🚀 12. Future Enhancements (Roadmap Ideas)

- 🤖 AI-powered moderation assistance
- 🔮 Predictive user engagement alerts
- 📱 Mobile Admin App
- 👮 Delegated family verifiers
- 🔗 Third-party integrations (email, SMS)

✅ 13. Summary

The **Administrator Module of My Parivaar** is designed to be:

✨ **Powerful** – Complete control & insights

🔐 **Secure** – Role-based, auditable, compliant

📈 **Scalable** – Grows with families & users

❤️ **Human-centric** – Built around trust & relationships

If you want next, I can:

- 📐 Create **Admin Module wireframes**
- 🧱 Define **database schema for admin entities**
- 🔄 Map **end-to-end admin workflows**
- 🏗️ Align this with **Supabase / Azure architecture**

Just say the word 👍

Perfect 👍\ I'll **proceed immediately with the next steps** and deliver **usable, concrete artifacts** you can plug into design & engineering.\ Since internal files do **not yet contain Admin wireframes / schemas / workflows**, the following are **clearly marked as *Proposed & Recommended* designs**, aligned with *My Parivaar* vision and scalable architecture (Supabase / Flutter / Azure-ready).

🚀 NEXT STEPS – ADMINISTRATOR MODULE (DETAILED)

🧭 1️⃣ Administrator Module – IA & Wireframe Blueprint 🧩

(Proposed – UX-ready)

🏠 Admin Landing (Dashboard)

```
┌──────────────
```

- Alert-driven workflow 🚨
- Read-first, act-later safety 🔐

👤 User Management – Wireframe Flow

Search: [Name | Email | Phone] ---------- 👤 Ravi Kumar ✅ Active 👤 Ananya Sharma ⚠️ Flagged 👤 Mohan Rao ❌ Disabled ---------- [View Profile] [Disable] [Audit Log]

🔍 **Profile View (Read-Only Sensitive Fields)**

- Basic Info
- Family Memberships
- Last Login
- Activity Timeline
- Reports Count

📄 Content Moderation – Review Queue

🚨 Flagged Content (14) ---------- 📷 Family Photo – Reported 📝 Post – Inappropriate Language 💬 Comment – Spam ---------- [Approve ✅] [Remove ❌] [Warn ⚠️]

🛡️ Every action → **mandatory reason capture**

🧱 2️⃣ Database Schema – Admin & Governance 📐

(Proposed – Engineering-ready)

👥 admin_users

```sql
id (uuid, pk)
name
email
role (super_admin | admin | moderator | support)
status (active | disabled)
created_at
last_login_at
```

🔐 admin_roles_permissions

```sql
role
permission_key
description
```

✅ Enables **fine-grained RBAC**

📃 audit_logs

```sql
id
actor_id (admin_users)
action_type
entity_type
entity_id
metadata (jsonb)
created_at
```

📌 **Non-deletable table** (compliance-safe)

🚨 content_reports

```sql
id
reported_by (user_id)
content_type
content_id
reason
status (pending | resolved | dismissed)
reviewed_by (admin_id)
review_notes
created_at
```

🎧 support_tickets

```sql
id
user_id
category
priority
status
assigned_to
internal_notes
created_at
resolved_at

```

🔄 3️⃣ End-to-End Admin Workflows 🔁

(Operational clarity)

🚨 Workflow A: Content Report Handling

1️⃣ User reports content\ 2️⃣ Entry created in `content_reports`\ 3️⃣ Moderator sees item in dashboard\ 4️⃣ Reviews content + history\ 5️⃣ Takes action:

- ✅ Approve

- ❌ Remove
- ⚠️ Warn user\

  6️⃣ Action logged in `audit_logs`\ 7️⃣ User notified automatically 🔔

✅ **Zero silent actions**

👤 Workflow B: User Suspension

1️⃣ Admin opens user profile\ 2️⃣ Reviews activity + reports\ 3️⃣ Clicks **Disable Account**\ 4️⃣ Mandatory reason entered\ 5️⃣ User status updated\ 6️⃣ All sessions invalidated\ 7️⃣ Audit log created

🔐 Security-first design

⚙️ Workflow C: Feature Toggle Rollout

1️⃣ Super Admin opens **Settings**\ 2️⃣ Enables feature for:

- All users
- Selected families
- Beta testers\

  3️⃣ Change propagated via config service\ 4️⃣ Logged + reversible

🚀 Enables **safe experimentation**

📌 4️⃣ What You Have Now ✅

You now have:

- ✅ **Admin IA & Wireframe blueprint**
- ✅ **Production-grade database schema**
- ✅ **Clear operational workflows**
- ✅ **Audit & compliance baked-in**

This is **ready for**:

- 🎨 UX wireframing (Figma)
- 🧑‍💻 Backend implementation (Supabase)
- 📱 Flutter admin screens
- 🔐 Enterprise hardening (Azure)

👉 Recommended Next (I can do immediately)

Choose any (or all):

1️⃣ 📐 **Figma-ready screen list + UX copy**\ 2️⃣ 🧮 **Supabase SQL scripts & RLS rules**\ 3️⃣ 🔐 **Security threat model for Admin Module**\ 4️⃣ 🤖 **AI-assisted moderation design**\ 5️⃣ 📊 **Admin analytics KPIs definition**

Just tell me what you want next ✌️

Below is a **Security Threat Model for the My Parivaar – Administrator Module**.

> ⚠️ **Grounding & Transparency**
>
> - I searched the available enterprise files for **existing security or threat-model documentation** related to *My Parivaar Admin Module*.
> - **No internal documents explicitly define a security threat model** for the Admin Module. >
>
> ✅ Therefore, the content below is **clearly marked as *Proposed & Recommended***, based on **industry-standard threat-modeling practices (STRIDE-aligned)** and tailored to the **My Parivaar Admin architecture** you have been designing.

🔐 My Parivaar – Administrator Module

Security Threat Model (Proposed)

🎯 1. Scope & Objectives

🔎 Scope

This threat model covers:

- Admin Console (Web)
- Admin Authentication & Authorization
- User / Family / Content Management
- Audit Logs & Configuration Settings
- Backend APIs supporting Admin actions

✅ Security Objectives

- Prevent **unauthorized admin access**
- Protect **sensitive family & user data**
- Ensure **accountability & traceability**
- Minimize **blast radius of admin misuse**
- Maintain **platform trust**

🧠 2. Threat Modeling Approach

We apply a **STRIDE-based model**:

	Category	Meaning
1		Spoofing Identity
2		Tampering with Data
3		Repudiation
4		Information Disclosure
5		Denial of Service
6		Elevation of Privilege

🏗️ 3. High-Level Admin Architecture (Security View)

```sql
Admin Browser
   |
   |  HTTPS + MFA
   |
Admin Frontend (RBAC enforced)
   |
   |  Signed JWT / Session Token
   |
Admin APIs
   |
   ├── User / Family Services
   ├── Content Moderation Service
   ├── Config Service
   └── Audit Log (Append-only)

```

🔐 **Trust boundaries exist at:**

- Browser → Frontend
- Frontend → Backend APIs
- APIs → Database

🕵️ 4. Threat Analysis & Mitigations (STRIDE)

🕵️ S – Spoofing Identity

Threats

- Stolen admin credentials
- Session hijacking
- Token replay attacks

Mitigations ✅

- Multi-Factor Authentication (MFA) mandatory for all admins
- Short-lived JWTs with refresh rotation

- IP & device fingerprinting for admin sessions
- Forced re-authentication for sensitive actions
- Admin login anomaly detection (geo, time)

🧨 T – Tampering with Data

Threats

- Unauthorized modification of user or family data
- Silent changes to system configurations
- Log manipulation

Mitigations ✅

- Role-based access control (RBAC) at API level
- Immutable audit logs (append-only)
- Field-level authorization checks
- Configuration changes require reason + confirmation
- Hash verification for critical config values

🧾 R – Repudiation

Threats

- Admin denies performing a destructive action
- Lack of traceability for moderation decisions

Mitigations ✅

- Every admin action logged with:
  - Admin ID
  - Timestamp
  - IP address
  - Action metadata
- Logs are non-editable & non-deletable
- Mandatory reason capture for:
  - User suspension
  - Content removal
  - Role changes

🔍 I – Information Disclosure

Threats

- Admin viewing data beyond role scope
- Leakage of PII (phone, email, family relations)
- Accidental data export

Mitigations ✅

- Least-privilege RBAC (Support ≠ Moderator ≠ Admin)
- Masking of sensitive fields by default

- Explicit permission for data export
- Watermarking exported reports
- Secure admin-only API endpoints

🚫 D – Denial of Service

Threats

- Abuse of admin APIs
- Bulk destructive operations
- Automated admin attacks

Mitigations ✅

- Rate limiting on admin APIs
- Bulk actions require confirmation + throttling
- Admin action quotas per session
- Circuit breakers on high-risk endpoints
- WAF rules for admin routes

⬆️ E – Elevation of Privilege

Threats

- Admin gaining Super Admin rights
- Misconfigured permissions
- Exploiting backend role checks

Mitigations ✅

- Role changes restricted to Super Admin only
- Dual-control (two-person rule) for role escalation
- RBAC enforced server-side (never frontend-only)
- Automated permission regression tests
- Alert on any privilege change

🛡️ 5. High-Risk Admin Actions & Controls

	Action	Risk	Control
1	Disable User	High	MFA + Reason + Log
2	Delete Content	Medium	Review Queue + Log
3	Change Config	High	Super Admin + Dual Approval
4	Assign Role	Critical	Dual Control + Alert
5	Export Data	High	Explicit Permission + Watermark

👁️ 6. Monitoring, Alerts & Detection

🚨 Real-Time Alerts

- Multiple failed admin logins
- Role change attempts
- Bulk actions
- Off-hours admin activity

📊 Continuous Monitoring

- Admin activity heatmaps
- Privilege usage review
- Dormant admin accounts detection

🔄 7. Incident Response (Admin-Specific)

1️⃣ Detect suspicious admin activity

2️⃣ Immediately lock admin account

3️⃣ Invalidate all sessions

4️⃣ Review audit logs

5️⃣ Rollback affected changes

6️⃣ Notify security owner

✅ Designed for **rapid containment**

✅ 8. Security Principles Enforced

- 🔐 Zero Trust
- 🧱 Defense in Depth
- 👤 Least Privilege
- 📜 Non-Repudiation
- 🔍 Observability by default

📌 9. Summary

This threat model ensures the **My Parivaar Admin Module** is:

✅ Secure against internal & external threats

✅ Auditable and compliance-ready

✅ Resistant to misuse & privilege abuse

✅ Scalable for future enterprise adoption

👉 Next steps I can deliver immediately:

1️⃣ 🔐 Security checklist for engineering 2️⃣ 🧪 Admin abuse test scenarios 3️⃣ 📜 Compliance mapping (GDPR / DPDP-India) 4️⃣ 🏗️ Zero-Trust reference architecture 5️⃣ 🤖 AI-based admin anomaly detection design

Just tell me what you want next 👍